DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	Naga Vaishnavi .R
ROLL NO	AV.SC.U4CSE25239
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-8
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	9-11
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	12-14
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	15-16
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	17-23

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10)Search data in SLL 11)Display or Traverse 12)Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	24-30
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	31-35
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	36-43
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	44-50

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	51-58
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	59-61
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	62-70

WEEK-1: -

Aim : A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by “Launch!”.

Code :

week1.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week1.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n <= 0:
        print("Rocket Launched")
    else:
        print(n)
        countdown(n - 1)
n = int(input("Enter the number: "))
countdown(n)
```

Output :

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week1.1.py ==========
Enter the number: 7
7
6
5
4
3
2
1
Rocket Launched
>>>
```

Aim : Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

Code :

week1.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week1.2.py (3.14.0)

File Edit Format Run Options Window Help

```
p = float(input("Enter the Principal Growth Factor:"))
n = int(input("Enter the Number of Years:"))
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)
Output = power(p, n)
print("Growth of an Investment", Output)
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week1.2.py ====
========
Enter the Principal Growth Factor:200
Enter the Number of Years:4
Growth of an Investment 1600000000.0
>>>
```

Aim : A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

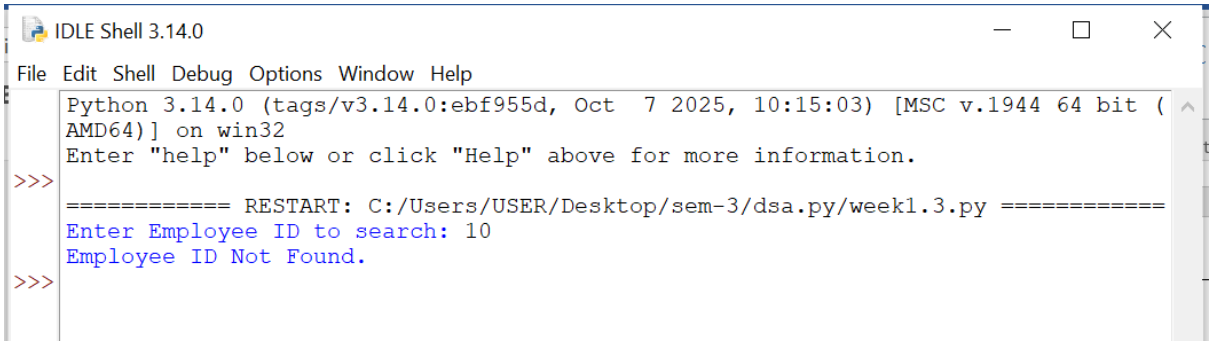
Code:

week1.3.py - C:/Users/USER/Desktop/sem-3/dsa.py/week1.3.py (3.14.0)

File Edit Format Run Options Window Help

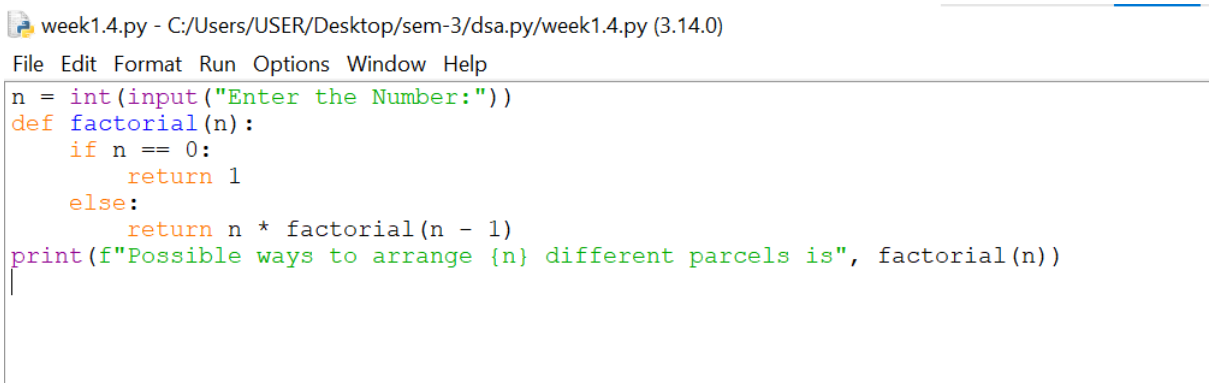
```
def employee(emp_list, target, index=0):
    if index == len(emp_list):
        return False
    if emp_list[index] == target:
        return True
    return employee(emp_list, target, index + 1)
employees = [510, 511, 512, 513, 514]
target_id = int(input("Enter Employee ID to search: "))
if employee(employees, target_id):
    print("Employee ID Found.")
else:
    print("Employee ID Not Found.")
```

Output :

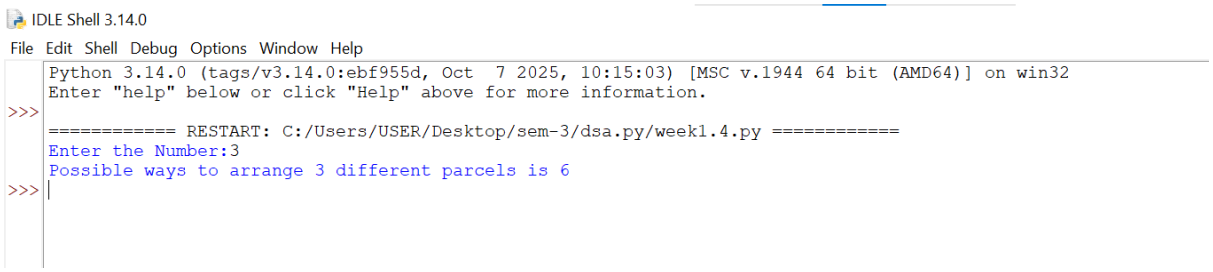


Aim: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code :

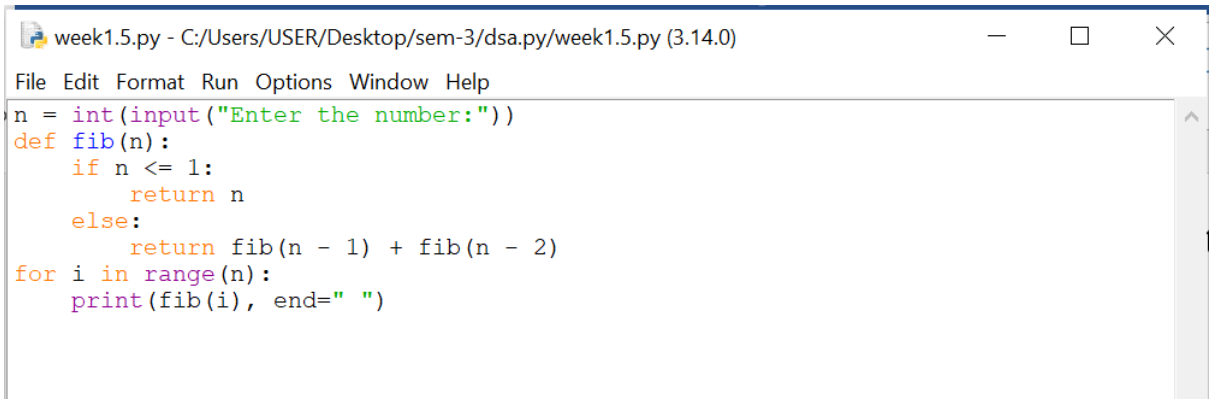


Output :

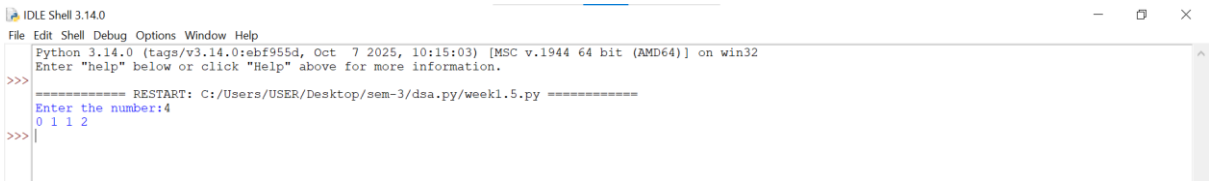


Aim : Write a recursive python program to print the first n terms of the Fibonacci series.

Code :



Output :



WEEK-2 :-

Aim : Write a python program to implement linear search.

Code :

week2.3.py - C:/Users/USER/Desktop/sem-3/dsa.py/week2.3.py (3.14.0)

File Edit Format Run Options Window Help

```
def linear_search(arr, target):
    """
    Performs a linear search on a list.
    Returns the index of the target if found, otherwise returns -1.
    """
    for index, element in enumerate(arr):
        if element == target:
            return index  # Target found, return current index

    return -1  # Target not found after checking the whole list


# Example Usage:
numbers = [10, 23, 45, 70, 11, 15]
target_value = 70

result = linear_search(numbers, target_value)

if result != -1:
    print(f"Element found at index: {result}")
else:
    print("Element not found in the list")
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week2.3.py ============
Element found at index: 3
>>>
```

Aim: Write a python program to implement binary search with and without sorted array as an input

Code :

With sorted array:

week2.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week2.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
n = int(input("Enter the number of elements:"))
arr = []
print("Enter elements in sorted order:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search:"))
result = binary_search(arr,key)
if result != -1:
  print(f"Element found at index {result}")
else:
    print("Element not found")
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week2.1.py ==========
Enter the number of elements:3
Enter elements in sorted order:
10
20
40
Enter the element to search:40
Element found at index 2
>>>
```

Without sorted array :

week2.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week2.2.py (3.14.0)

File Edit Format Run Options Window Help

```
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
n = int(input("Enter the number of elements:"))
arr = []
print("Enter elements in sorted order:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search:"))

if arr == sorted(arr):
    print("The input list is already sorted.")
else:
    print("The input list is not sorted.")
    print("The list is sorting...")
    arr.sort()
print(f"Sorted list: {arr}")

result = binary_search(arr,key)
if result != -1:
  print(f"Element found at index {result}")
else:
    print("Element not found")
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week2.1.py ============
Enter the number of elements:3
Enter elements in sorted order:
10
20
40
Enter the element to search:40
Element found at index 2
>>>
```

WEEK-3:-

Aim : Write a python program to implement bubble sort.

Code :

week3.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week3.1.py (3.14.0)

File Edit Format Run Options Window Help

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Bubble Sort algorithm
for i in range(n - 1):
    for j in range(n - i - 1):
        # Change '<' to '>' for ascending order
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Elements after Bubble Sort (Ascending):")
print(arr)
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week3.1.py ============
Enter the number of elements: 3
Enter the elements:
10
20
1
Elements after Bubble Sort (Ascending):
[1, 10, 20]
>>> A
```

Aim :Write a python program to implement insertion sort.

Code:

week3.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week3.2.py (3.14.0)

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    n = len(arr)
    for i in range (1,n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j = j - 1
        arr[j+1] = key
    return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    element = int(input())
    arr.append(element)
result = insertion_sort(arr)
print("Elements after Insertion Sort:")
print(result)
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week3.2.py ============
Enter the number of elements: 5
Enter the elements:
23
11
453
65
91
Elements after Insertion Sort:
[11, 23, 65, 91, 453]
>>>
```

Aim : Write a python program to implement selection sort.

Code:

week3.3.py - C:/Users/USER/Desktop/sem-3/dsa.py/week3.3.py (3.14.0)

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range (i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
        arr[i],arr[min_index]=arr[min_index],arr[i]
    return arr
n=int(input("Enter the number of elements: "))
arr=[]
print("Enter the elements:")
for i in range(n):
    element=int(input())
    arr.append(element)
result=selection_sort(arr)
print("Elements after Selection Sort:")
print(result)
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week3.3.py ============
Enter the number of elements: 4
Enter the elements:
29
12
3
24
Elements after Selection Sort:
[3, 12, 24, 29]
>>>
```

WEEK-4:-

Aim: Write a python program to implement quick sort.

Code :

week4.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week4.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def quicksort(a,low,high):
    if low < high:
        i = low
        j = high
        pivot = low
        while i < j:
            while i < len(a) and a[i]<=a[pivot]:
                i +=1
            while a[j] > a[pivot]:
                j = j -1
            if i < j:
                a[i],a[j] = a[j],a[i]
        a[j],a[pivot] = a[pivot],a[j]
        quicksort(a,low,j-1)
        quicksort(a,j+1,high)
a = list(map(int,input("Enter numbers to sort:").split(" ")))
n = len(a)
quicksort(a,0,n-1)
print("Sorted array:")
print(a)
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week4.1.py ============
Enter numbers to sort:12 45 31 23 4
Sorted array:
[4, 12, 23, 31, 45]
>>>
```

Aim : Write a python program to implement merge sort.

Code :

week4.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week4.2.py (3.14.0)

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid=len(arr)//2
    left=merge_sort(arr[:mid])
    right=merge_sort(arr[mid:])
    i = j = k = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
    return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    element = int(input())
    arr.append(element)
result = merge_sort(arr)
print("Elements after Merge Sort:")
print(result)
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week4.2.py ============
Enter the number of elements: 3
Enter the elements:
12
13
43
Elements after Merge Sort:
[12, 13, 43]
>>>
```

WEEK-5:-

Aim :Write a python program to implement a Singly Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Sort the data of SLL
10)Search data in SLL
11)Display or Traverse
12)Exit

Code :

week5.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week5.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def create(self, n):
        self.head = None

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            self.insert_end(data)

        print("Linked List created successfully")

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new

    def insert_position(self, data, position):
        new = Node(data)

        if position < 1:
            print("Invalid position")
```

week5.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week5.1.py (3.14.0)

File Edit Format Run Options Window Help

```
            temp.next = new

    def insert_position(self, data, position):
        new = Node(data)

        if position < 1:
            print("Invalid position")
            return

        if position == 1:
            new.next = self.head
            self.head = new
            print("Node inserted successfully")
            return
        temp = self.head

        for i in range(position - 2):
            if temp is None:
                print("Invalid position")
                return
            temp = temp.next

        if temp is None:
            print("Invalid position")
        else:
            new.next = temp.next
            temp.next = new
            print("Node inserted successfully")

    def delete_value(self, value):
        if self.head is None:
            print("No Data to delete")
            return

        if self.head.data == value:
            self.head = self.head.next
            print("Value deleted")
            return

        temp = self.head

        while temp.next and temp.next.data != value:
```

week5.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week5.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")

    def delete_first(self):
        if self.head is None:
            print("No Data to delete")
        else:
            print("Deleted Value =", self.head.data)
            self.head = self.head.next

    def delete_last(self):
        if self.head is None:
            print("No Data to delete")

        elif self.head.next is None:
            print("Deleted Value =", self.head.data)
            self.head = None

        else:
            temp = self.head

            while temp.next.next:
                temp = temp.next

            print("Deleted Value =", temp.next.data)
            temp.next = None

    def count(self):
        if self.head is None:
            print("No Linked List")
        else:
            c = 0
            temp = self.head

            while temp:
```

week5.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week5.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        if self.head is None:
            print("No Linked List")
        else:
            c = 0
            temp = self.head

            while temp:
                c += 1
                temp = temp.next

            print("Number of nodes =", c)

    def display(self):
        if self.head is None:
            print("No Linked List")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")

ll = LinkedList()

while True:
    print("\n--- Singly Linked List Operations ---")
    print("1. Create a Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Position")
    print("5. Delete a Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Nodes")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
```

week5.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week5.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        n = int(input("Enter number of nodes: "))
        ll.create(n)

    elif choice == 2:
        data = int(input("Enter data: "))
        ll.insert_begin(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        ll.insert_end(data)

    elif choice == 4:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        ll.insert_position(data, position)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        ll.delete_value(value)

    elif choice == 6:
        ll.delete_first()

    elif choice == 7:
        ll.delete_last()

    elif choice == 8:
        ll.count()

    elif choice == 9:
        ll.display()

    elif choice == 10:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
9. Display Nodes
10. Exit
Enter your choice: 1
Enter number of nodes: 10
Enter data for node 1: 10
Enter data for node 2: 9
Enter data for node 3: 8
Enter data for node 4: 7
Enter data for node 5: 6
Enter data for node 6: 5
Enter data for node 7: 5
Enter data for node 8: 4
Enter data for node 9: 3
Enter data for node 10: 1
Linked List created successfully

--- Singly Linked List Operations ---
1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit
Enter your choice: 9
10 -> 9 -> 8 -> 7 -> 6 -> 5 -> 5 -> 4 -> 3 -> 1 -> None

--- Singly Linked List Operations ---
1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit
Enter your choice:
```

WEEK-6:

Aim :Write a python program to implement a Double Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Display or Traverse
10) Exit

Code:

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp
        print("Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
        print("Node inserted at beginning.")

    def insert_end(self):
```

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node inserted at end.")

    def insert_position(self):
        position = int(input("Enter position: "))
        data = int(input("Enter data: "))

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            new_node = Node(data)

            if self.head is not None:
                new_node.next = self.head
                self.head.prev = new_node

            self.head = new_node
            print("Node inserted.")
            return

        temp = self.head

        for i in range(1, position - 1):
            if temp is None:
                print("Invalid position.")
```

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        temp = self.head

        for i in range(1, position - 1):
            if temp is None:
                print("Invalid position.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid position.")
            return

        new_node = Node(data)

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted.")

    def delete_beginning(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Node deleted from beginning.")

    def delete_end(self):
        if self.head is None:
            print("Linked List is empty.")
            return
```

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        temp = self.head

        if temp.next is None:
            self.head = None
            print("Node deleted from end.")
            return

        while temp.next is not None:
            temp = temp.next

        temp.prev.next = None

        print("Node deleted from end.")

    def delete_position(self):
        position = int(input("Enter position: "))

        if self.head is None:
            print("Linked List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.head = self.head.next

            if self.head is not None:
                self.head.prev = None

            print("Node deleted.")
            return

        temp = self.head

        for i in range(1, position):
            if temp is None:
                print("Invalid position.")
                return
            temp = temp.next
```

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        for i in range(1, position):
            if temp is None:
                print("Invalid position.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid position.")
            return

        if temp.next is not None:
            temp.next.prev = temp.prev

        if temp.prev is not None:
            temp.prev.next = temp.next

        print("Node deleted.")

    def count(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next
        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("Linked List is empty.")
            return
        temp = self.head
        print("Doubly Linked List:")
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

dll = DoublyLinkedList()
```

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    print("1. Create a LL")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Deletion at beginning")
    print("6. Deletion at end")
    print("7. Deletion at specific position")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_position()

    elif choice == 5:
        dll.delete_beginning()

    elif choice == 6:
        dll.delete_end()

    elif choice == 7:
        dll.delete_position()

    elif choice == 8:
        dll.count()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
```

week6.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    print("6. Deletion at end")
    print("7. Deletion at specific position")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_position()

    elif choice == 5:
        dll.delete_beginning()

    elif choice == 6:
        dll.delete_end()

    elif choice == 7:
        dll.delete_position()

    elif choice == 8:
        dll.count()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Outputt :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week6.1.py ============

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 10
Enter data: 20
Enter data: 30
Linked List created successfully.

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Doubly Linked List:
10 <-> 20 <-> 30 <-> None

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
```

Activate Windows
Go to Settings to activate Window

WEEK-7:-

Aim :Write a python program to implement stack operations such as push, pop, peek, display using an array.

Code:

week7.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week7.2.py (3.14.0)

File Edit Format Run Options Window Help

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])


size = int(input("Enter size of stack: "))
s = StackArray(size)

while True:
```

week7.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week7.2.py (3.14.0)

File Edit Format Run Options Window Help

```
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])


size = int(input("Enter size of stack: "))
s = StackArray(size)

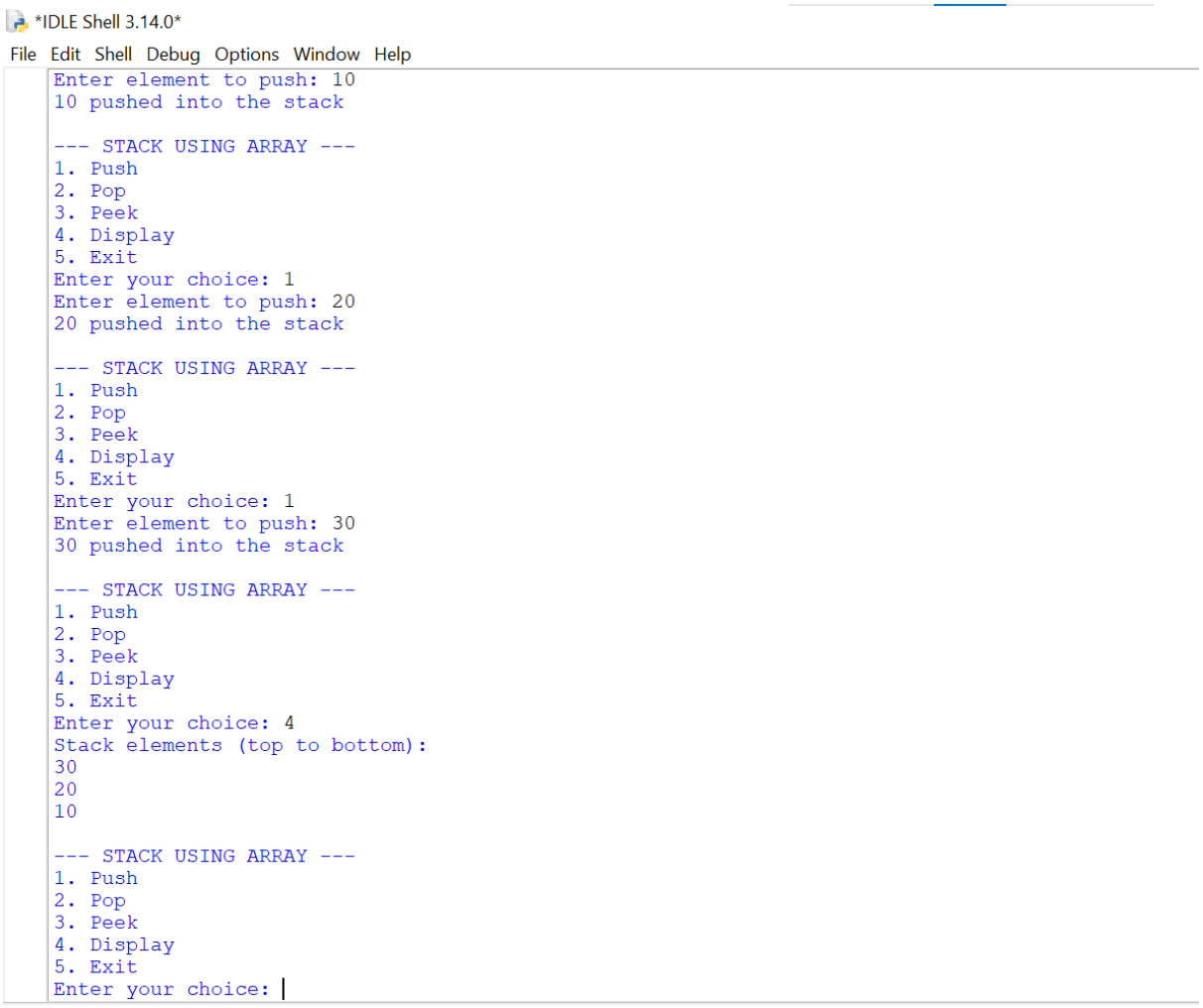
while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

Act
Go t

Output :



Aim :Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

Code :

week7.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week7.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            popped = self.top.data
            self.top = self.top.next
            print(popped, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next

sll = StackLinkedList()
```

week7.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week7.1.py (3.14.0)

File Edit Format Run Options Window Help

```
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next

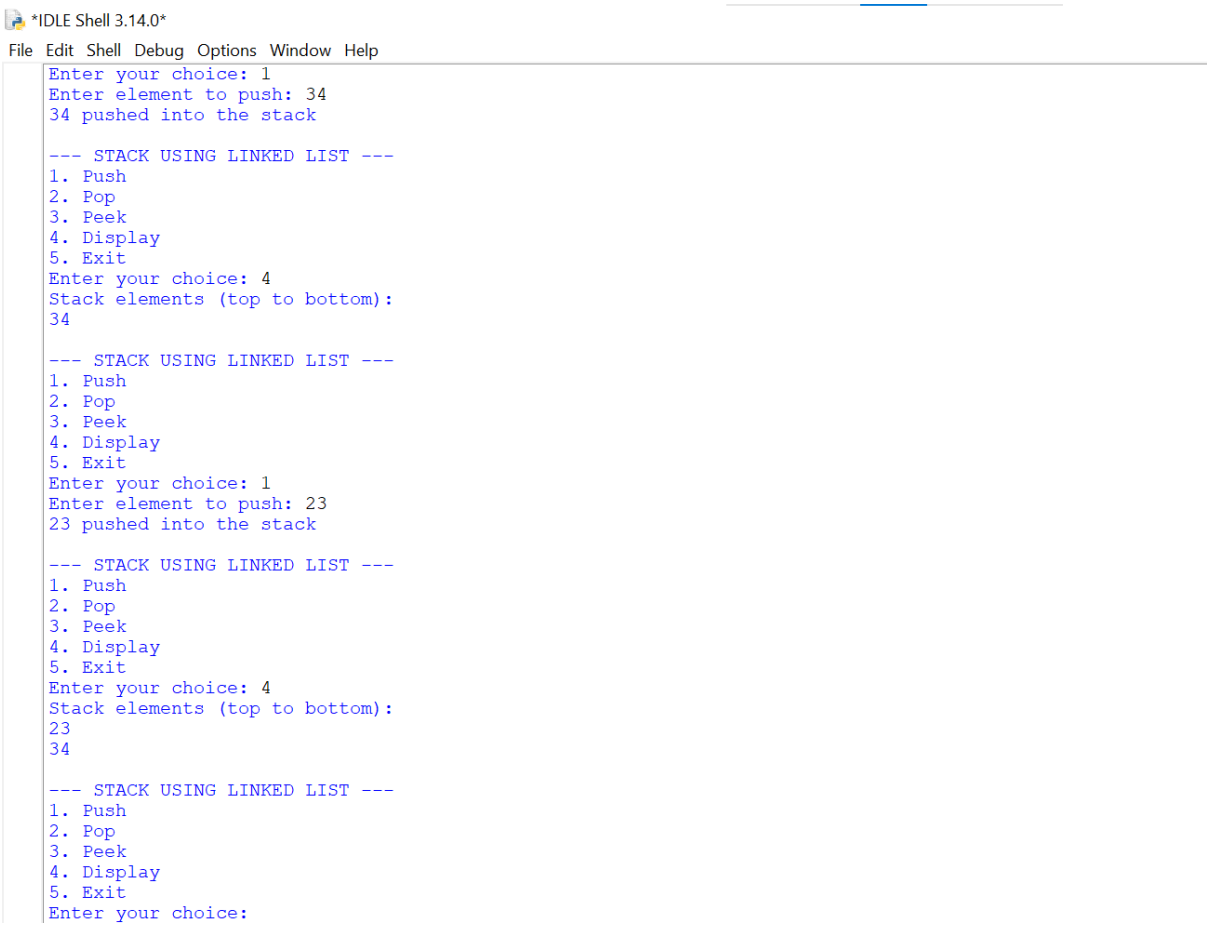
sll = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        sll.push(item)
    elif choice == 2:
        sll.pop()
    elif choice == 3:
        sll.peek()
    elif choice == 4:
        sll.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter your choice: 1
Enter element to push: 34
34 pushed into the stack
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
34
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 23
23 pushed into the stack
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
23
34
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:

WEEK-8:-

Aim :Write a python program to implement Queue and its operations such as en-queue, de-queue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

Code :

Using Arrays :

week8.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week8.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
```

week8.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week8.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
>>>
============ RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week8.1.py ============
Enter the size of Queue: 2

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 1
1 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
1
2

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

Using LinkedLists :

week8.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week8.2.py (3.14.0)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None

            print(f"{x} deleted from the queue")
            self.front += 1

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
```

week8.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week8.2.py (3.14.0)

File Edit Format Run Options Window Help

```
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
        print("\n----- Queue Menu -----")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            item = int(input("Enter the element to enqueue: "))
            q.enqueue(item)

        elif choice == 2:
            q.dequeue()

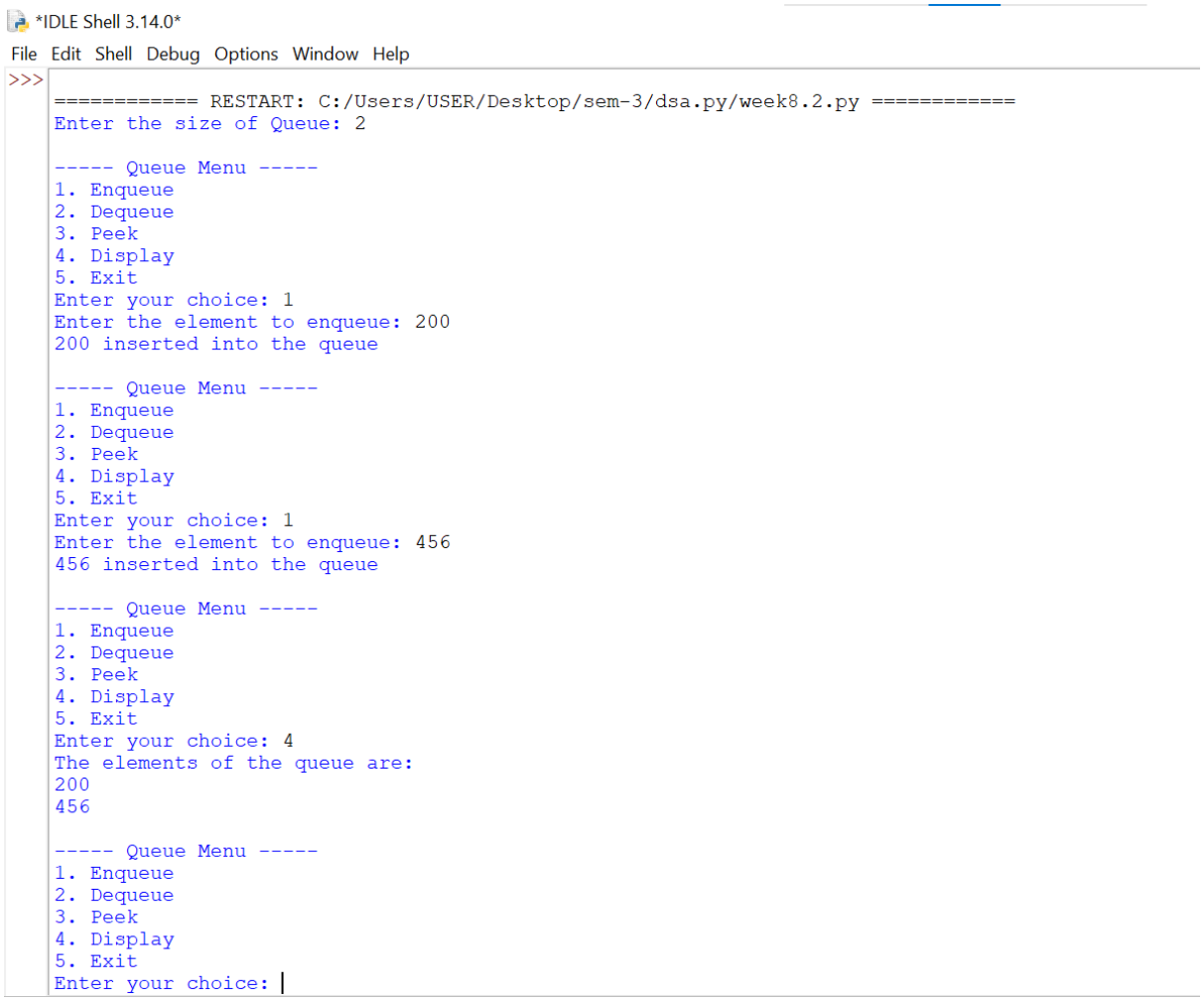
        elif choice == 3:
            q.peek()

        elif choice == 4:
            q.display()

        elif choice == 5:
            print("Exiting Program")
            break

        else:
            print("Invalid Choice!")
```

Output :



Aim :Write a python program to implement Circular Queue and its operations such as en-queue, de-queue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

Code :

week8.3.py - C:/Users/USER/Desktop/sem-3/dsa.py/week8.3.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self, size=5):
        self.size = size
        self.front = None
        self.rear = None
        self.count = 0

    def enqueue(self, x):
        if self.count == self.size:
            print("Queue Overflow")
        else:
            new_node = Node(x)

            if self.front is None:
                self.front = new_node
                self.rear = new_node
                new_node.next = new_node
            else:
                new_node.next = self.front
                self.rear.next = new_node
                self.rear = new_node

            self.count += 1
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            x = self.front.data

            if self.front == self.rear:
                self.front = None
                self.rear = None
            else:
                self.front = self.front.next
                self.rear.next = self.front
```

week8.3.py - C:/Users/USER/Desktop/sem-3/dsa.py/week8.3.py (3.14.0)

File Edit Format Run Options Window Help

```
            else:
                self.front = self.front.next
                self.rear.next = self.front

            self.count -= 1
            print(f"{x} deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")

            temp = self.front

            while True:
                print(temp.data)
                temp = temp.next

                if temp == self.front:
                    break


size = int(input("Enter the size of Queue: "))
q = CircularQueue(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter the element to enqueue: 40
40 inserted into the queue
----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 432
432 inserted into the queue
----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 23
23 inserted into the queue
----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
40
432
23
----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:

WEEK-9 :

Aim :Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals – in-order, pre-order and post-order
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

Code :

week9.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
```

week9.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1
```

week9.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1

def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
```

week9.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)

def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
```

week9.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
```

week9.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week9.1.py ==========
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 36
Element inserted successfully.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter your choice: 52
Invalid choice. Please try again.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 45
Element inserted successfully.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 14
Program terminated.
>>>

WEEK-10:-

Aim : Write a menu driven python program to implement an AVL Tree and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

Code :

week10.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:
    def height(self, root):
        if root is None:
            return 0
        return root.height

    def get_balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        return x

    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
```

week10.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        return y

    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left rotate(root)
```

week10.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        balance = self.get_balance(root)

        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def search(self, root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    def find_min(self, root):
        while root.left is not None:
            root = root.left
        return root

    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
```

week10.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
```

week10.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

tree = AVLTree()
root = None

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Element found.")
        else:
```

week10.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Element found.")
        else:
            print("Element not found.")

    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(root)

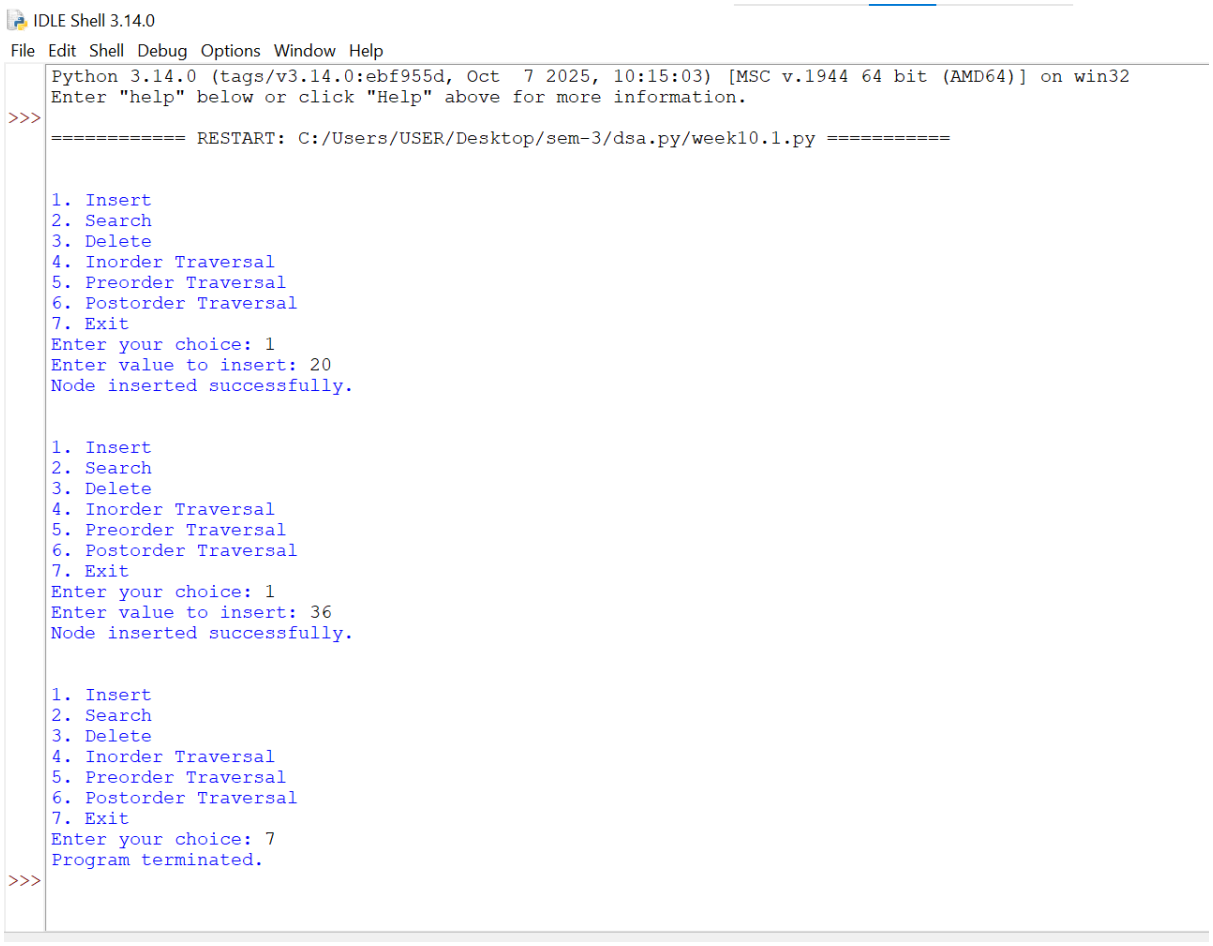
    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(root)

    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week10.1.py ==========
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 20
Node inserted successfully.
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 36
Node inserted successfully.
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 7
Program terminated.
>>>

WEEK-11:-

Aim : Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

1. Insertion of an element into the Heap.
2. Deletion of the root element from the Heap.
3. Display the array representation of the Heap.
4. Exit the program.

Code :

week11.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week11.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class BinaryHeap:
    def __init__(self, heap_type="min"):
        self.heap = []
        self.heap_type = heap_type.lower()

    def _compare(self, parent_val, child_val):
        if self.heap_type == "max":
            return parent_val < child_val  # Max Heap: Parent should be >= Child
        else:
            return parent_val > child_val  # Min Heap: Parent should be <= Child

    def insert(self, key):
        self.heap.append(key)
        self._heapify_up(len(self.heap) - 1)
        print(f"Successfully inserted {key}.")

    def delete_root(self):
        if not self.heap:
            print("Heap underflow! The heap is currently empty.")
            return None

        root_val = self.heap[0]
        last_val = self.heap.pop()

        if self.heap:
            self.heap[0] = last_val
            self._heapify_down(0)

        print(f"Successfully deleted root node ({root_val}).")
        return root_val

    def _heapify_up(self, index):
        parent = (index - 1) // 2
        while index > 0 and self._compare(self.heap[parent], self.heap[index]):
            self.heap[parent], self.heap[index] = self.heap[index], self.heap[parent]
            index = parent
            parent = (index - 1) // 2

    def _heapify_down(self, index):
        size = len(self.heap)
        extreme_index = index
        left = 2 * index + 1
```

week11.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week11.1.py (3.14.0)

File Edit Format Run Options Window Help

```
    def _heapify_down(self, index):
        size = len(self.heap)
        extreme_index = index
        left = 2 * index + 1
        right = 2 * index + 2

        # Check if left child breaks the heap property
        if left < size and self._compare(self.heap[extreme_index], self.heap[left]):
            extreme_index = left

        # Check if right child breaks the heap property
        if right < size and self._compare(self.heap[extreme_index], self.heap[right]):
            extreme_index = right

        # Swap and continue heapifying if a child broke the rule
        if extreme_index != index:
            self.heap[index], self.heap[extreme_index] = self.heap[extreme_index], self.heap[index]
            self._heapify_down(extreme_index)

    def display(self):
        """Displays the array representation of the heap."""
        if not self.heap:
            print("The heap is empty.")
        else:
            print(f"{self.heap_type.capitalize()} Heap Array: {self.heap}")


def main():
    print("--- Binary Heap Implementation ---")
    while True:
        heap_choice = input("Choose Heap Type:\n1. Min Heap\n2. Max Heap\nChoice (1 or 2): ").strip()
        if heap_choice == '1':
            h = BinaryHeap(heap_type="min")
            break
        elif heap_choice == '2':
            h = BinaryHeap(heap_type="max")
            break
        else:
            print("Invalid selection. Please enter 1 or 2.\n")

    while True:
        print(f"\n======== {h.heap type.upper()} HEAP OPERATIONS ========")
```

week11.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week11.1.py (3.14.0)

File Edit Format Run Options Window Help

```
        heap_choice = input("Choose Heap Type:\n1. Min Heap\n2. Max Heap\nChoice (1 or 2): ").strip()
        if heap_choice == '1':
            h = BinaryHeap(heap_type="min")
            break
        elif heap_choice == '2':
            h = BinaryHeap(heap_type="max")
            break
        else:
            print("Invalid selection. Please enter 1 or 2.\n")

    while True:
        print(f"\n======== {h.heap_type.upper()} HEAP OPERATIONS ========")
        print("1. Insertion of an element")
        print("2. Deletion of the root element")
        print("3. Display the array representation")
        print("4. Exit program")

        choice = input("Enter your choice (1-4): ").strip()

        if choice == '1':
            try:
                val = int(input("Enter an integer value to insert: "))
                h.insert(val)
            except ValueError:
                print("Error: Please enter a valid integer.")

        elif choice == '2':
            h.delete_root()

        elif choice == '3':
            h.display()

        elif choice == '4':
            print("Exiting application. Goodbye!")
            break

        else:
            print("Invalid input! Please choose an option between 1 and 4.")

if __name__ == "__main__":
    main()
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
>>>
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week11.1.py ==========
--- Binary Heap Implementation ---
Choose Heap Type:
1. Min Heap
2. Max Heap
Choice (1 or 2): 1
======== MIN HEAP OPERATIONS ========
1. Insertion of an element
2. Deletion of the root element
3. Display the array representation
4. Exit program
Enter your choice (1-4): 1
Enter an integer value to insert: 10
Successfully inserted 10.
======== MIN HEAP OPERATIONS ========
1. Insertion of an element
2. Deletion of the root element
3. Display the array representation
4. Exit program
Enter your choice (1-4): 1
Enter an integer value to insert: 20
Successfully inserted 20.
======== MIN HEAP OPERATIONS ========
1. Insertion of an element
2. Deletion of the root element
3. Display the array representation
4. Exit program
Enter your choice (1-4): 3
Min Heap Array: [10, 20]
======== MIN HEAP OPERATIONS ========
1. Insertion of an element
2. Deletion of the root element
3. Display the array representation
4. Exit program
Enter your choice (1-4): 4
Exiting application. Goodbye!
>>>

WEEK-12:-

Aim : Write a program to implement the following three collision resolution techniques in a Hash Table:

1. Separate Chaining
2. Linear Probing
3. Quadratic Probing

Code :

Linear Probing

week12.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week12.1.py (3.14.0)

File Edit Format Run Options Window Help

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")


def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
```

week12.1.py - C:/Users/USER/Desktop/sem-3/dsa.py/week12.1.py (3.14.0)

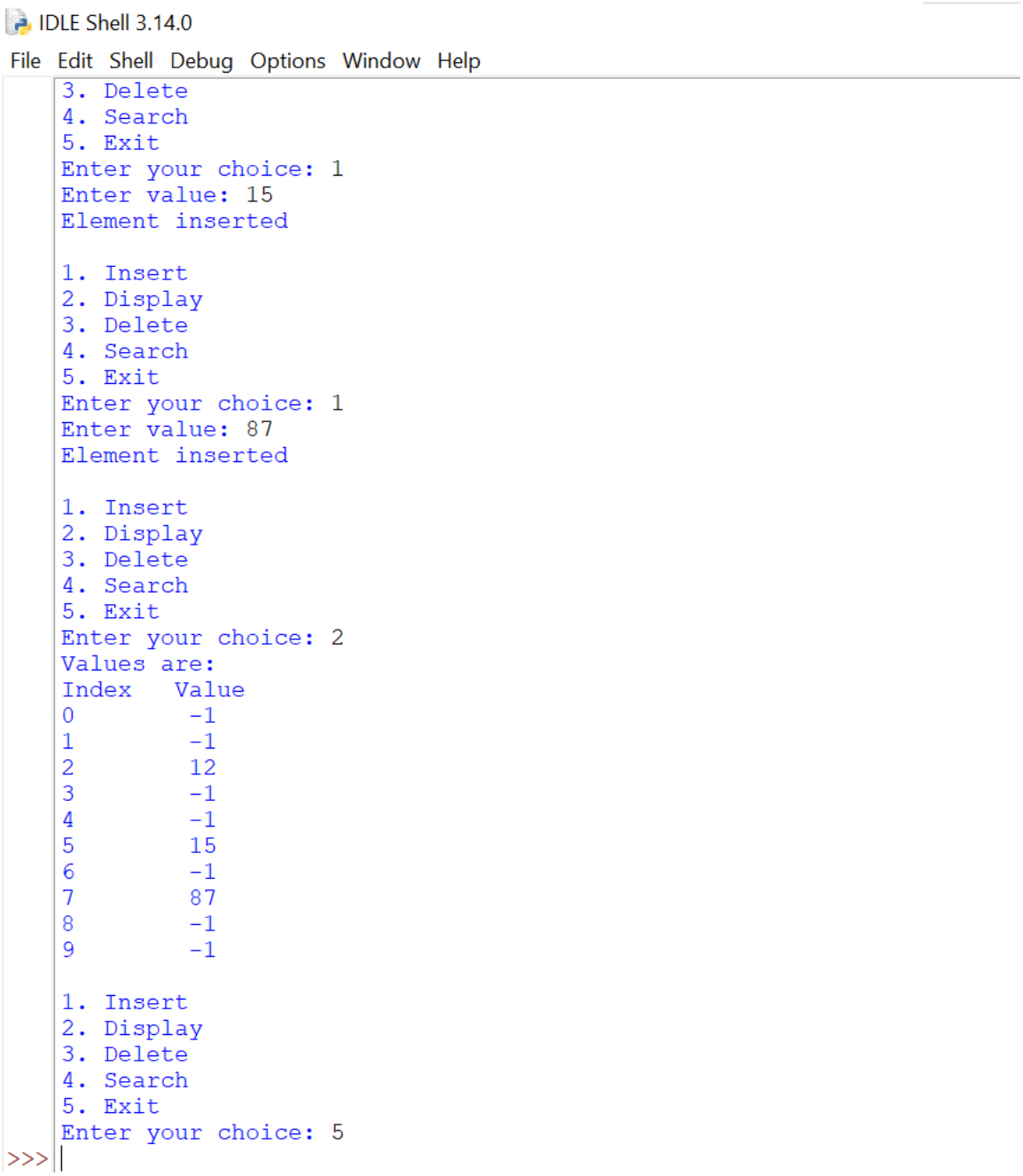
File Edit Format Run Options Window Help

```
def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")


def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
    elif ch == 5:
        repeat = False
    else:
        print("Invalid choice")
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 15
Element inserted
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 87
Element inserted
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 2
Values are:
Index Value
0 -1
1 -1
2 12
3 -1
4 -1
5 15
6 -1
7 87
8 -1
9 -1
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 5
>>>

Separate Chaining :

Code :

week12.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week12.2.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE


def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode


def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
```

week12.2.py - C:/Users/USER/Desktop/sem-3/dsa.py/week12.2.py (3.14.0)

File Edit Format Run Options Window Help

```
def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode


def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

Output :

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
========== RESTART: C:/Users/USER/Desktop/sem-3/dsa.py/week12.2.py ==========

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 10

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 20

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 30

1. Insert
2. Display
3. Exit
Enter your choice: 2

Hash table values:
0: 10 -> 20 -> 30 -> None
1: None
2: None
3: None
4: None
5: None
6: None
7: None
8: None
9: None

1. Insert
2. Display
3. Exit
Enter your choice: 3
>>>
```

Quadratic Probing :

Code :

week12.3.py - C:/Users/USER/Desktop/sem-3/dsa.py/week12.3.py (3.14.0)

File Edit Format Run Options Window Help

```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")


def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

Output :


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 12
Element inserted at index 3
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 23
Element inserted at index 4
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 23
No empty slot found in the probe sequence
1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
-1
-1
-1
12
23
-1
-1
-1
-1
-1
1. Insert
2. Display
3. Exit
Enter your choice: 3
>>>